

Retrieval-Augmented Generation, commonly known as RAG, combines information retrieval with large language models.

RAG enables language models to access external knowledge sources during inference.

Traditional LLMs rely solely on their training data.

RAG allows models to retrieve information from documents that were not present during training.

This improves factual accuracy and reduces hallucinations.

Large Language Models are neural networks trained on vast amounts of text data.

LLMs learn patterns, relationships, and structures in language.

Popular LLMs include GPT, Claude, Llama, and Mistral.

LLMs generate responses based on probability distributions.

They predict the most likely next token in a sequence.

Tokenization is the process of breaking text into smaller units called

Tokens may represent words, subwords, or characters.

Different models use different tokenization methods.

Tokenization is the first step in most NLP pipelines.

Every LLM has a maximum context window measured in tokens.

Efficient tokenization helps optimize model performance.

Token counts influence API costs.

Long documents often require chunking before processing.

Chunking divides large text into manageable pieces.

Chunking ensures information fits within model limits.

Chunk size can significantly impact retrieval quality.

Small chunks provide precision but may lose context.

Large chunks preserve context but may reduce relevance.

Overlapping chunks help maintain continuity between sections.

Chunk overlap is a common strategy in RAG systems.

Text splitters automate chunk creation.

Recursive text splitters are widely used in LangChain.

Metadata is often attached to each chunk.

Metadata may include source, title, page number, and author.

Metadata helps with filtering and traceability.

Embeddings convert text into numerical vector representations.

Embeddings capture semantic meaning rather than exact keywords.

Similar meanings often produce similar vectors.

Embedding models transform text into high-dimensional vectors.

A vector may contain hundreds or thousands of dimensions.

OpenAI provides popular embedding models.

Sentence Transformers are widely used open-source embedding models.

Cohere also offers embedding services.

BGE embeddings are popular for retrieval applications.

Embeddings power semantic search systems.

Semantic search focuses on meaning rather than exact word matches.

It enables more intelligent information retrieval.

Similarity search compares vectors mathematically.

Cosine similarity is a common similarity metric.

Euclidean distance is another similarity measure.

Dot product calculations are frequently used in vector search.

Vector similarity helps identify relevant content.

Query embeddings are compared with document embeddings.

The most similar vectors are retrieved.

These retrieved vectors form the basis of context retrieval.



Vector databases are designed to store and search embeddings efficiently.

Traditional databases are not optimized for vector similarity search.

Vector databases enable fast nearest-neighbor retrieval.

They support large-scale AI applications.

Modern RAG systems rely heavily on vector databases.

Vector databases can store vectors and metadata together.

They support filtering and ranking operations.

Vector indexes improve retrieval speed.

Approximate nearest-neighbor algorithms enhance performance.

Scalability is a major advantage of vector databases.

Chroma is a popular open-source vector database.

Chroma integrates well with LangChain.

Chroma is easy for beginners to learn.

It can run locally without cloud infrastructure.

Chroma stores embeddings and associated metadata.

Chroma supports similarity search operations.

Chroma is commonly used in tutorials and prototypes.

Developers often choose Chroma for learning RAG.

Chroma provides persistent storage options.

Chroma works with multiple embedding providers.

Pinecone is a managed cloud vector database.

Pinecone simplifies vector storage and retrieval.

It is designed for production-scale applications.

Pinecone provides high availability and scalability.

Many enterprise AI solutions use Pinecone.

Pinecone supports metadata filtering.

Pinecone offers low-latency retrieval.

It integrates with popular AI frameworks.

Pinecone helps manage billions of vectors.

Pinecone is widely adopted in industry.

FAISS stands for Facebook AI Similarity Search.

FAISS was developed by Meta AI.

FAISS is an open-source vector search library.

It enables fast similarity search on large datasets.

FAISS supports CPU and GPU acceleration.

FAISS is commonly used in research projects.

Many vector databases use concepts similar to FAISS.

FAISS supports multiple indexing strategies.

It can perform exact and approximate searches.

FAISS is a cornerstone technology in vector retrieval.

During query processing, user questions are converted into embeddings.

Query embeddings are compared with stored document vectors.

The top matching chunks are retrieved.

Retrieved chunks become contextual knowledge.

The context is inserted into the prompt.

This process is called prompt augmentation.

The LLM receives both the question and retrieved context.

The model generates a grounded response.

RAG improves accuracy, transparency, and relevance.

RAG has become a foundational architecture for modern Generative AI applications.

LangChain helps build LLM applications